

CONFIDENTIAL — CANDIDATE ASSESSMENT

Software Development Technical Assessment

SUSPICIOUS FILE SUBMISSION & VIRUSTOTAL ANALYSIS PORTAL

Prepared by ElevateX · Wessel Swanepoel

Developer recruitment — technical evaluation

Version 2 (working draft) · 26 August 2026

Document Control

Document	Software Development Technical Assessment
Prepared by	Wessel Swanepoel, ElevateX (Pty) Ltd
Purpose	Practical evaluation of a candidate's engineering judgment, verification discipline, and decision-making in an AI-assisted development workflow
Version	2.0 (working draft)
Classification	Confidential — for shortlisted candidates only
Distribution	Issued individually to candidates under assessment

Revision History

Version	Date	Author	Change
1.0	01 June 2025	Wessel Swanepoel	Original assessment brief
2.0	26 August 2026	Wessel Swanepoel	Evolved for AI-assisted development: real-world API constraints promoted to core requirements, verification & testing requirements added, <code>DECISIONS.md</code> deliverable introduced, live working session defined, prescriptive technology choices relaxed to .NET (C#), insights dashboard added

This document and the scenario it describes are the property of ElevateX (Pty) Ltd and are shared with you solely for the purpose of this assessment. Please do not redistribute or publish the brief itself.

Contents

1. Introduction — How This Assessment Works	4
2. Scenario	5
3. Functional Requirements	6
4. Verification & Testing	8
5. Working with AI Tools	9
6. DECISIONS.md — Your Decision Log	9
7. Technical Guidelines	10
8. The Live Working Session	11
9. What to Submit	12
10. Questions	12

1. Introduction — How This Assessment Works

Thank you for your interest in the opportunity with us. This assessment simulates a real development task and is designed to evaluate the skills that matter in a modern, AI-assisted engineering workflow: architectural judgment, problem-solving under real-world constraints, a verification mindset, data reasoning, and the ability to explain and defend your decisions.

We assume you will use AI development tools — we encourage it. Because AI has made *producing* working code fast and cheap, this assessment doesn't stop at "does it run". It is structured in three parts, and the later parts carry as much weight as the code itself:

Part	What	What we're looking at
Part A	The build (take-home)	A working prototype built to the requirements in §3, including how your design handles the real-world constraints in §2
Part B	Written artifacts (take-home)	Your tests (§4) and your decision log, <code>DECISIONS.md</code> (§6) — these are first-class deliverables, not appendices
Part C	Live working session (60–90 min)	A structured session where we walk through your submission together, extend it live, and discuss your reasoning (§8)

Across all three parts, we assess the same dimensions:

Dimension	What good looks like
Architecture & decomposition	A design that fits the problem's size — clear boundaries, sensible layering, no speculative scaffolding
Correctness under constraints	Behaves correctly when the API rate-limits, calls fail, submissions burst, or the app restarts
Verification mindset	Deliberate choices about what to test and why; tests that would actually catch a regression
Data reasoning	A data model that fits the domain, and a dashboard that answers the questions the business would actually ask of it
Communication & judgment	Decisions you can explain, trade-offs you can name, and alternatives you considered and rejected
AI leverage	Using AI tools effectively <i>and</i> critically — knowing when the output is wrong and overriding it

WHAT STANDS OUT

The strongest submissions are rarely the ones with the most features. They are the ones where every part of the system has a reason to exist, and the candidate can explain that reason. Depth beats breadth.

2. Scenario

Our team is building an internal web application to help security analysts submit potentially malicious files for scanning using the **VirusTotal API**. Analysts upload files with contextual metadata, return later to review submission statuses, and view scan results once analysis completes.

Two facts about how this system is used shape the design:



- Submissions arrive in **bursts** — an analyst triaging a phishing campaign may submit 20 attachments at once.
- Analysts **frequently resubmit files that have already been analysed** — the same sample often arrives from multiple mailboxes.

REAL-WORLD CONSTRAINT — THE VIRUSTOTAL PUBLIC API

You will use a **free public VirusTotal API key** (sign up at virustotal.com). The free tier is a scarce, unreliable resource, and your design must treat it as one:

- **4 requests per minute** and **500 requests per day** — exceeding this returns HTTP 429
- **32 MB** standard upload limit
- Transient failures and slow analysis queues are normal, not exceptional
- Files uploaded to VirusTotal become part of its corpus and are **accessible to other researchers** — worth keeping in mind for an app whose users handle sensitive material

DESIGN ACCORDINGLY

A naive implementation that fires an API call per submission will fall over the moment an analyst submits a burst of files. How your design absorbs bursts, respects the rate limit, spends its daily quota wisely, and recovers from failure **is the core of this assessment** — it is where we will spend most of the live session.

3. Functional Requirements

Build a working prototype on **.NET (C#)**. That is the only technology constraint — the UI framework, persistence approach, and background-processing design are **architecture decisions we expect you to make**, and to motivate in `DECISIONS.md` (§6). Requirements are prioritised MoSCoW-style: **MUST** items are the core task, **SHOULD** items are where submissions differentiate themselves, and **COULD** items are optional stand-outs.

ID	Requirement	Priority
FR-01	Upload with metadata. Accept any file type. Prompt for the <i>source</i> of the file (e.g. email, USB, download) and the <i>reason for suspicion</i> ; add any further metadata fields you think the domain needs.	MUST
FR-02	Local persistence. Submissions and results persist locally and survive restarts. Your choice of store is yours to motivate, with one hard rule: it self-provisions on first run — cloning and running the app requires no database server, no SQL scripts, and no manual setup steps from us. 	MUST
FR-03	Submissions list. Show all submissions with file name, SHA-256, submission date, status (Queued → In Progress → Completed / Failed), and the scan summary once available. 	MUST
FR-04	Background processing. Scans are processed asynchronously in the background — the UI never blocks on VirusTotal, and uploading and browsing stay responsive while scans are in flight. How you implement this (hosted service, channels, a queue abstraction) is an architecture decision to motivate 	MUST
FR-05	Rate-limit compliance. The dispatcher never exceeds 4 API requests per minute. A burst of 20 submissions queues cleanly and drains at a compliant pace.	MUST
FR-06	Failure handling. Retry transient failures (429s, timeouts, 5xx) with backoff. After bounded retries, a submission lands in a visible <code>Failed</code> state with a reason — never a silent hang. 	MUST
FR-07	Results display. For completed scans, show the number of engines that flagged the file out of the total, a brief summary, and a link to the full VirusTotal report.	MUST
FR-08	Duplicate awareness. When a file with a SHA-256 the system has already processed is resubmitted, link it to the existing analysis rather than spending quota re-scanning it.	SHOULD
FR-09	Live status updates. Status changes reach the UI without a manual refresh. Simple polling is perfectly acceptable.	SHOULD
FR-10	Restart resilience. Queued and in-progress submissions survive an application restart and resume processing.	SHOULD

ID	Requirement	Priority
FR-11	Insights dashboard. A dashboard view giving statistical insight into submissions over time, presented in a visually considered way — e.g. submission volume and detection rates over time, status breakdown, top sources of suspicious files, API quota consumption. Choose aggregations and chart types that answer questions an analyst or manager would actually ask 	SHOULD
FR-12	Stand-outs. SignalR real-time updates · <code>.xlsx</code> export of submission data · structured logging and error notifications · a considered strategy for files over 32 MB 	COULD

HINT

Before writing any code, read the VirusTotal API documentation and think about what a single "scan" actually costs you in requests. The API offers more than one way to learn about a file. Designs that spend the quota thoughtfully stand out.

ON THE DASHBOARD (FR-11)

In our client work, the dashboard is the deliverable people remember. It is also where we see how you think about data: which aggregates matter, which visualisation fits each question, and whether the numbers are actually correct. A small, accurate, well-chosen dashboard beats a wall of charts.

4. Verification & Testing

AI can generate code quickly; the engineering skill is knowing whether it is *correct*. We require automated tests — but we care far more about **which tests you chose** than how many you wrote.

ID	Requirement	Priority
VT-01	Test the logic that matters. Unit tests cover the interesting behaviour: rate-limited dispatch, retry/backoff, the status state machine, duplicate detection. UI markup and trivial CRUD plumbing do not need tests.	MUST
VT-02	Isolate the external dependency. The VirusTotal client sits behind an interface and is faked or stubbed in tests. Your test suite runs offline, without an API key, in seconds.	MUST
VT-03	Declare your boundaries. In <code>DECISIONS.md</code> , state what you deliberately did <i>not</i> test and why. A considered boundary is a feature of a good test suite, not a gap 	MUST
VT-04	One integration test. A single end-to-end test proving a submission flows from upload through the (faked) scan pipeline to a completed status.	SHOULD

A NOTE ON AI-GENERATED TESTS

A small number of sharp tests beats broad, shallow coverage. Tests that assert nothing meaningful — or that merely restate the implementation — count *against* a submission, not for it. If an AI tool generated your tests, review them as critically as you would a stranger's pull request; we will.

5. Working with AI Tools

Use whatever AI tooling you normally work with — Copilot, Claude, Cursor, ChatGPT, or none at all. There is no penalty for heavy AI use and no bonus for abstaining. Driving these tools well is now part of the job, and we want to see how you do it.

Three ground rules:

- **You own every line.** In the live session we will open files *we* choose and ask why the code is the way it is. "The AI did it that way" is a fine origin story but not an explanation — you should have had a reason to keep it.
- **Keep short notes as you go.** Where did AI genuinely accelerate you? Where was it wrong, and what did you override? Two or three honest examples in `DECISIONS.md` (§6) tell us more than a polished feature list.
- **Verify, don't trust.** The requirements in §2 and §3 contain details a code generator will get wrong unless you steer it. Finding and fixing those is the exercise.

6. DECISIONS.md — Your Decision Log

Alongside your README, submit a `DECISIONS.md` of roughly 10–15 bullet points. This is a first-class deliverable — it is your written case for the design, and we read it before we read the code. Cover

- **Key decisions and rejected alternatives** — the 3–5 choices that shaped the system (queue design, schema, state handling, project structure) and what you decided *against*, and why
- **Scale honesty** — what breaks first if submission volume grows 10x, and what you would change
- **With more time** — what you would do differently or add next, in priority order
- **AI usage** — where AI tools helped most, where they were wrong or suboptimal, and what you overrode
- **Testing boundaries** — what you deliberately did not test, and why (VT-03)

WHY WE ASK

Working code tells us what you built. `DECISIONS.md` tells us how you think. Candidates who write "I chose X, considered Y, and rejected it because Z" are demonstrating exactly the judgment we are hiring for — even when we would have chosen Y.

7. Technical Guidelines

- The solution must be built on **.NET (C#)**. Within that, the stack is yours: UI framework (Blazor, Razor Pages, MVC, ...), persistence, and background-processing approach are architecture decisions — make them deliberately, and motivate them in `DECISIONS.md` 
- Whatever storage you choose must **self-provision on first run** — running your submission requires no database server, SQL scripts, migrations, or manual setup steps from us (FR-02)
- Keep the API key and environment-specific settings in `appsettings.json` (or user secrets) — **do not commit a real API key** to the repository; note in your README how we should supply ours
- **No authentication, registration, or user management** — assume a trusted internal application in a secure environment
- A simple, clear, responsive UI is all that's needed — starting from a default project template (yes, even the ugly purple one) is encouraged

7.1 Time expectation

Plan for around **4–6 focused hours**. With AI assistance, the basic build comes together quickly — spend the time you save on the parts that differentiate: the queue and failure design, the dashboard, the tests, and `DECISIONS.md`. We are not expecting production-grade code; we are assessing your thought process, architectural decisions, and ability to build working, maintainable features. Please stop at a sensible point rather than gold-plating — "what I'd do next" belongs in `DECISIONS.md`, not in a 20-hour submission. 

8. The Live Working Session

Within a few days of your submission we will schedule a 60–90 minute working session — in person or over a screen-share. This is not an interrogation; it is the kind of session you would have with a colleague on your first week. It has four parts:

Segment	~Time	What happens
1. Walkthrough	15 min	We open files <i>we</i> pick and talk through them. Why is this here? What happens if the API times out right there? What does this state transition protect against?
2. Live extension	25 min	We introduce a new requirement on the spot and you implement or sketch it in your own codebase. You are welcome to use your AI tools live — how you prompt, and whether you verify what comes back, is part of what we're watching.
3. Code & test critique	15 min	We show you a short piece of code or a test suite (not yours) and ask you to review it: what's wrong, what's missing, what would you change before merging?
4. Data reasoning	10 min	Questions against your own data model and dashboard — e.g. "how many distinct files from email sources were flagged by more than five engines this month?" Which of these can your dashboard answer today, how would your model answer the rest, and where does it creak?

HOW TO PREPARE

There are no tricks here. A candidate who built their submission and understands it needs no preparation at all. If parts of your codebase were AI-generated and you haven't read them yet — read them before the session. That's the whole secret.

9. What to Submit

ID	Deliverable	Priority
D-01	A public GitHub repository (preferred) or downloadable ZIP	MUST
D-02	README.md — setup instructions (including how to supply an API key), how to run the tests, notes and assumptions, and your favourite Punk, Emo or Hard-Rock band 😊	MUST
D-03	DECISIONS.md — your decision log, per §6	MUST
D-04	A commit history that reflects how the work actually unfolded — small, honest commits beat one squashed "initial commit"	SHOULD

Running your submission should require nothing from us beyond cloning, supplying an API key, and `dotnet run`. No SQL files, no migrations, no extra database setup steps.

10. Questions

If anything is unclear, or you'd like to clarify scope, reach out any time: charlie@elevatex.tech.

Asking a sharp scoping question before you start is a signal in your favour, not against you. We're excited to see your work!

Thank you

ELEVATEX (PTY) LTD · ADVISORY & SOLUTIONS

BUSINESS DEVELOPMENT & ACCOUNTS

Stefan Oberholzer
082 887 9979
stefan@elevatex.tech

Ruan Dickinson
072 393 1579
ruan@elevatex.tech

Wessel Swanepoel
082 405 8973
wessel@elevatex.tech

ADDRESS

Summit Place, Building 6
Garsfontein Road, Menlyn
Pretoria, South Africa

REGISTRATION NUMBER

2015/001727/07

VAT NUMBER

4350274058